THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE
CS-4031 Compiler Construction — Fall 2026

Lecture 2
Structure of a Compiler: Phases Overview; Front-/Middle-/Back-End; MLIR and Real-World IRs

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§1.2 (The Structure of a Compiler), §2.1–§2.8 (overview of a syntax-directed translator's stages)
Other references:	[PP] Programming Language Pragmatics §1.5, §1.6

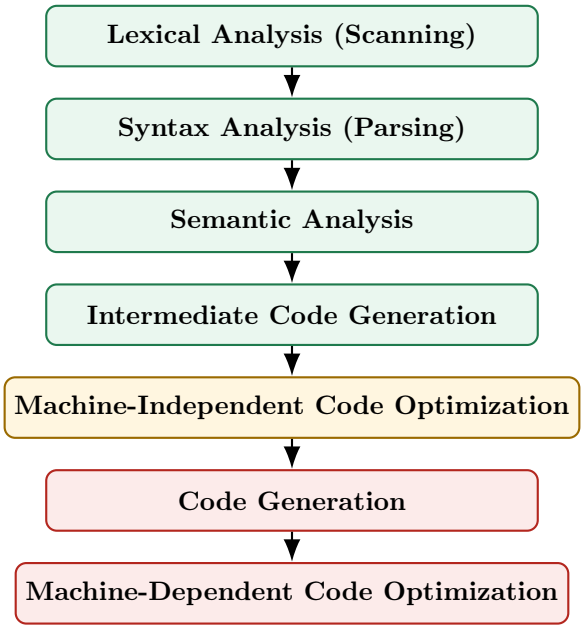
Lecture Roadmap

1. The classical phase pipeline of a compiler (DB Fig. 1.6): what each phase consumes and produces, followed all the way through on one tiny worked example.
2. The two cross-cutting components every phase touches: the **symbol table** and **error handling**.
3. Grouping the phases into **front-end** / **middle-end** / **back-end**, and why that grouping is an engineering decision, not just a diagram convenience.
4. How real-world intermediate representations – **LLVM IR, JVM bytecode, WebAssembly, and MLIR** – relate to this pipeline and to each other.

Today is still "the outside view." From Lecture **??** onward we go phase-by-phase in depth, starting with lexical analysis. (The preprocessor/assembler/linker/loader toolchain around the compiler was already covered in Lecture **??**.)

2.1 The Phase Pipeline of a Compiler

[DB] DB organizes a compiler as a pipeline of phases, each of which takes the output of the previous phase as input, checks it, and produces a new representation of the program one step closer to executable code.



Phase	Input	Output
Lexical Analysis	Raw character stream	Stream of **tokens** (e.g. `ID`, `NUM`, `+`)
Syntax Analysis	Token stream	**Syntax tree** (or parse tree) capturing grammatical structure
Semantic Analysis	Syntax tree	Annotated ("decorated") tree: types checked, scopes resolved
Intermediate Code Gen.	Annotated tree	Machine-independent **IR**, e.g. three-address code
Code Optimization	IR	Improved, semantically-equivalent IR
Code Generation	Optimized IR	Target-machine code (often assembly)

Exam Focus

Memorize the phase order and, for each phase, one sentence describing its input and output. This exact table (input → phase → output) is the single most common short-answer / MCQ pattern in compiler exams worldwide, not just here.

2.1.1 A Tiny Example, Followed Through Every Phase

[DB] It helps enormously to see one small piece of source code survive the *entire* pipeline, phase by phase, before we study each phase in isolation over the coming lectures. DB uses exactly this running example when it first introduces the phases (§1.2); we reproduce a miniature version of it here, and will point back to it by name ("the `position` example") whenever a later lecture revisits one of these phases in more depth. Nothing here needs to be memorized yet – it is a preview, not new material to be tested on its own.

Source statement (assume `position`, `initial`, and `rate` are declared as real/floating-point variables, and `60` is an ordinary integer literal):

```
position = initial + rate * 60
```

Definition: Token and Lexeme

A **lexeme** is the actual sequence of characters in the source program that the lexical analyzer recognizes as a single meaningful unit – for example, the seven characters `p-o-s-i-t-i-o-n` form one lexeme, and the single character `=` forms another. A **token** is the abstract `<token-name, attribute-value>` pair the lexer emits once it has recognized a lexeme: the token-name records *which category* was matched (e.g. `ID`, `ASSIGN`, `PLUS`), and the attribute-value carries whatever extra information later phases need in order to tell this particular lexeme apart from other lexemes in the same category – for an identifier, almost always a pointer into the symbol table. Lecture **??** gives the full formal treatment, including **patterns** – the rules that describe which lexemes belong to which token.

For this statement, the **lexemes** are the individual pieces of source text `position`, `=`, `initial`, `+`, `rate`, `*`, and `60`. Lexical analysis classifies each one as a **token**: the lexeme `position` becomes `<ID, ptr to symtab entry for position>` – written `<id,1>` in the walkthrough below – and likewise `initial` and `rate` become `<id,2>` and `<id,3>`; the lexeme `=` becomes an `<ASSIGN>` token, `+` a `<PLUS>` token, `*` a `<MULT>` token, and `60` a `<NUM, 60>` token. None of the three operator tokens need an attribute-value, since the token-name alone already tells the parser everything it needs to know.

Example: The `position` Statement, Phase by Phase

- **Lexical analysis.** The character stream is grouped into tokens: `<id,1> <=> <id,2> <+> <id,3> <*> <60>` – where `id,1`, `id,2`, and `id,3` are pointers to the symbol-table entries

for `position`, `initial`, and `rate` respectively.

- **Syntax analysis.** The token stream is organized into a syntax tree that reflects the grammatical structure of an assignment: an `=` node whose left child is `id,1` and whose right child is a `+` node, which in turn has children `id,2` and a `*` node with children `id,3` and `60`.
- **Semantic analysis.** Types are checked, and where necessary, made explicit. Here, `60` is an integer literal but `rate` is real, so the semantic analyzer inserts an explicit conversion: the `*` node's right child becomes `inttoreal(60)` instead of the bare integer `60`. The result is an *annotated* ("decorated") tree.
- **Intermediate code generation.** The annotated tree is translated into three-address code, one operation per instruction:

```
t1 = inttoreal(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

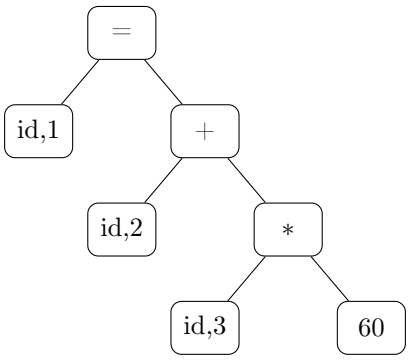
- **Machine-independent code optimization.** `inttoreal(60)` does not depend on any variable, so it can be computed once, at compile time, instead of being recomputed by the generated program every single time this line runs:

```
t1 = id3 * 60.0
id1 = id2 + t1
```

- **Code generation.** The optimized IR becomes target assembly, using real machine registers and instructions (illustrative syntax, not tied to one specific ISA):

```
MOVF   id3, R2
MULF   #60.0, R2
MOVF   id2, R1
ADDF   R2, R1
MOVF   R1, id1
```

The syntax-analysis step above builds the following syntax tree – the root `=` node assigns to `id,1` the value computed by its right subtree:



Syntax tree for ***`position = initial + rate * 60.`***

Exam Focus

This example is deliberately small enough to hold in your head. When a later lecture introduces the *real* algorithm for a phase (Thompson's construction, LALR(1) table construction, available-expressions data-flow analysis, and so on), try re-running that algorithm on this same statement

by hand before tackling a harder one – it is a fast way to sanity-check that you have understood a new algorithm correctly.

2.1.2 Analysis and Synthesis: The Two Halves

[DB] DB groups these six phases into two conceptual halves:

- **Analysis part ("front end"):** lexical, syntax, and semantic analysis, plus intermediate code generation. This half *breaks down* the source program and builds an intermediate representation plus a record of the program's structure (the symbol table). Analysis is where most **compile-time errors** are caught and reported.
- **Synthesis part ("back end"):** optimization and code generation. This half *constructs* the desired target program from the intermediate representation and symbol table.

Definition: Analysis-Synthesis Model

A compiler can be understood as consisting of an **analysis** part, which breaks the source program into pieces and builds an intermediate representation, and a **synthesis** part, which constructs the target program from that intermediate representation. This is DB's own high-level characterization of *every* compiler, regardless of source or target language.

2.2 Symbol Table and Error Handling: The Cross-Cutting Phases

[DB] Two components are not steps in the pipeline — they run *alongside every phase*, being read from and written to throughout the whole compilation: the **symbol table** and **error handling**.

Definition: Symbol Table

A **symbol table** is a data structure that records every identifier declared in a program – variables, function names, class names – together with the attributes later phases need to know about it: its **type**, **scope**, **memory location**, and, for functions, its parameter and return types. It is not itself one of the six phases; instead every phase reads from it and writes to it: lexical analysis creates an entry the first time it sees a new identifier, semantic analysis fills in type information once it is known, and code generation consults it to compute final memory addresses.

Example: Symbol Table Contents for the `position` Example

For `position = initial + rate * 60` (§2.1.1), lexical analysis creates one entry per distinct identifier – exactly the `id,1`, `id,2`, and `id,3` pointers used throughout that walkthrough:

Entry	Name	Type	Role in this statement
`id,1`	`position`	real	assigned to
`id,2`	`initial`	real	read
`id,3`	`rate`	real	read

Notice `60` never gets its own entry – it is a numeric *literal*, not an identifier, so it is carried as a token attribute (`<NUM, 60>`) rather than being recorded in the symbol table.

Error handling. Each phase may encounter errors specific to its own level: lexical analysis catches malformed tokens, syntax analysis catches grammatically invalid programs, semantic analysis catches type mismatches and undeclared identifiers. A well-built compiler tries to *recover* from an error and keep going, so it can report several distinct errors from a single compilation run instead

of stopping at the first one.

2.2.1 Lexical, Syntax, and Semantic Errors

[DB] The three kinds of errors above are easy to name but easy to blur together in practice. Getting the distinction right matters because it tells you *which phase* is responsible for catching a given mistake – and, just as importantly, which phases are *not* responsible for it and will happily let it through.

Definition: Lexical, Syntax, and Semantic Errors

- A **lexical error** occurs when the lexer meets a character sequence that matches *no pattern for any token at all* – e.g. an illegal character, an unterminated string or comment, or a malformed numeric literal. The lexer fails before the parser ever sees this part of the input.
- A **syntax error** occurs when the token stream – with every individual token itself perfectly valid – does not conform to the grammar: e.g. a missing semicolon, an unbalanced parenthesis, or an operator with no right-hand operand.
- A **semantic error** occurs when the program is grammatically well formed but violates a rule about *meaning* that the language enforces: e.g. using an undeclared identifier, assigning incompatible types, or calling a function with the wrong number of arguments.

Each is caught by a different phase, in this order, because each phase can only see what the phase before it has already built: lexical analysis sees individual characters, syntax analysis sees the shape of the token stream, and semantic analysis is the first phase that understands what the program actually *means* (using the symbol table above to check declarations, types, and scope).

Example: One Error of Each Kind, in One Function

```
int main() {
    int count = 0;
    int limit = 10           /* (1) */

    while (count < limit) {
        count = count # 1;   /* (2) */
        total = total + count;   /* (3) */
    }

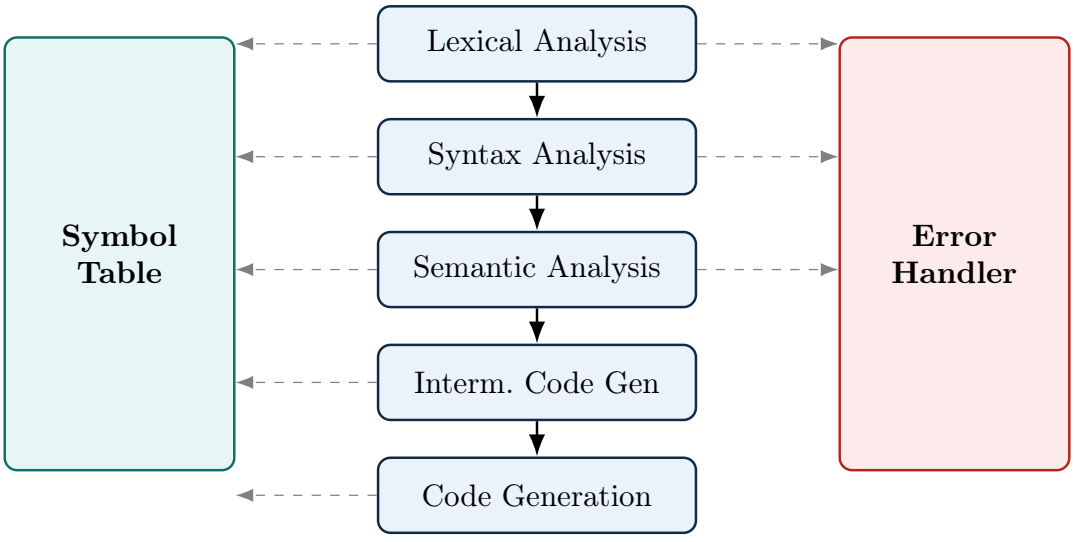
    printf("Final count = %d", count);
    return 0;
}
```

Line	Error Kind	Why
(1)	Syntax	`int limit = 10` is missing the terminating `;`. Every token on this line is individually a valid token – the parser is the one that fails, because it cannot complete the grammar rule for a declaration statement without a semicolon before the next statement begins.
(2)	Lexical	`#` is not a valid character inside a C expression. No token pattern matches it at all, so the *lexer itself* fails right here – the parser never even gets a chance to see this line.
(3)	Semantic	`total` is used but was never declared anywhere in this function. Every token is valid and the statement is a grammatically perfect assignment, so only semantic analysis – which checks the symbol table for declarations and scope – can catch this.

Notice the ordering this implies: if line (2)'s illegal character were fixed but line (1)'s missing semicolon were not, the compiler would still stop at the syntax error before ever reaching semantic analysis and reporting the undeclared `total` on line (3) – each phase only gets to run once the phase before it has accepted its input.

Exam Focus

Given a short, broken code snippet, be able to classify each mistake as lexical, syntax, or semantic, and justify the classification the way the table above does – this is one of the most common short-answer formats for this topic. A frequent trap: a misspelled *keyword* (like `fi` for `if`) is **not** a lexical error, because it is still a perfectly valid lexeme for token `ID` – the mistake only surfaces later, as a syntax error (if `fi` appears somewhere no identifier is grammatically valid) or a semantic error (if it is used as if it were a declared function or variable).



The symbol table is read and written by every phase, but only lexical, syntax, and semantic analysis report to the error handler – those are the three error categories this course teaches (§2.2.1). Intermediate code generation and code generation run only once the tree is already fully checked and error-free, so they are not shown wired to the error handler here.

Extra Depth: What Real Symbol Tables Store

A production compiler's symbol table stores far more than DB's introductory description suggests: mangled names (for overload resolution in C++/Rust), calling-convention information, alignment and padding requirements, visibility/linkage (`static` vs. exported), inlining hints, and — in languages with generics/templates — a separate table per instantiation. Modern compilers often implement it not as one flat table but as a **chain of nested scopes** (a scope stack or scope tree), so that entering a block, function, or class pushes a new scope that shadows outer ones and is popped on exit. LLVM's `clang` keeps this information in its `Decl` hierarchy rather than a single table, reflecting how much richer real symbol-table designs are compared to the simple table DB introduces.

2.3 Grouping the Phases: Front-End, Middle-End, Back-End

[beyond DB] DB's own diagram groups phases into analysis/synthesis, but the vocabulary you will hear constantly in industry and in later lectures is **front-end** / **middle-end** / **back-end**. This is not just relabeling — it reflects a real engineering strategy.

Group	Contains / Responsibility
Front-end	Lexical, syntax, and semantic analysis, plus intermediate code generation. Language-*specific*: knows the grammar and type rules of exactly one source language, and lowers it to a common IR.
Middle-end	Machine-independent optimization on the IR. Language-*agnostic* and target-*agnostic*: the same optimization passes (constant folding, dead-code elimination, inlining) apply no matter which source language or target CPU is involved.
Back-end	Code generation and machine-dependent optimization. Target-*specific*: knows the instruction set, register file, and calling convention of exactly one target architecture.

Why This Grouping Is the Whole Point

If front-ends only ever talk to a shared IR, and back-ends only ever talk to that same shared IR, then supporting a new *source language* only requires writing one new front-end — it automatically gets every optimization and every target the IR already supports. Symmetrically, supporting a new *target CPU* only requires writing one new back-end — it automatically works for every source language already supported. Without this split, you would need to write a separate optimizer and code generator for every (language, target) *pair* — the classic "M languages $\times$ N targets" problem, needing on the order of $M \times N$ components instead of $M + N$.

A Bit of History

This $M \times N$ argument predates LLVM by decades. In the 1960s, **T-diagrams** (popularized by F. L. Bauer and others) were a standard notation for exactly this problem: a T-shaped diagram showing a compiler written in language H, translating source language S into target language T. Compiler writers used T-diagrams to reason about *bootstrapping* (compiling a new compiler using an existing one) and about how many compilers you actually need to support M languages on N machines — precisely the front-end/back-end decoupling argument above, expressed sixty years earlier with pen-and-paper diagrams instead of shared IR software.

Extra Depth: The Front/Middle/Back Split in a Real Toolchain — LLVM

The clearest concrete instance of this architecture today is LLVM, and it is worth walking through once here since we will return to it in Lecture 25 for code generation proper.

- **Front-end (Clang, for C/C++/Objective-C):** lexes and parses source into a Clang-specific Abstract Syntax Tree (AST), performs semantic analysis and type checking on that AST, then *lowers* it into **LLVM IR** — a typed, SSA-based (Static Single Assignment) intermediate representation that is completely independent of C++ as a language. Rust (`rustc`), Swift, Julia, and Zig all have their own, completely different front-ends, but they all emit the *same* LLVM IR.
- **Middle-end (the LLVM optimizer, `opt`):** a pipeline of dozens of independent **passes** — e.g. `mem2reg` (promote stack variables to SSA registers), `instcombine` (peephole-simplify instruction sequences), `inline` (function inlining), `gvn` (global value numbering, subsuming common subexpression elimination), `loop-vectorize` — each of which reads LLVM IR and produces improved LLVM IR. None of these passes know or care whether the original source was C++, Rust, or Swift.
- **Back-end (`llc`, or the integrated code generator):** lowers optimized LLVM IR to a specific target's machine code — x86-64, AArch64, RISC-V, or even WebAssembly, selected via a `-target` triple. Instruction selection, register allocation, and instruction scheduling all happen here, specific to that one target.

The practical payoff: when Apple Silicon (AArch64) needed a good C++ compiler, no one had to write a new C++ front-end — Clang's C++ front-end and every middle-end optimization pass were already reusable; only a back-end for AArch64 was needed (and it mostly already existed, since LLVM had supported ARM for years). This is exactly the $M + N$ payoff from the coreidea box above, made concrete: roughly 20+ front-ends (Clang, `rustc`, Swift, Julia, ...) and roughly 20+ back-ends (x86-64, AArch64, RISC-V, WebAssembly, ...) share *one* middle-end, instead of each of those 20×20 combinations needing its own hand-written optimizer.

Extra Depth: The Front-End's Second Career — IDEs and Language Servers

The front-end (lexer + parser + semantic analyzer) is valuable even when no target code is ever generated. This observation underlies the **Language Server Protocol (LSP)**: an editor like VS Code or Neovim talks to a background process (a "language server") that re-runs a language's front-end on every keystroke to provide autocomplete, jump-to-definition, and real-time error squiggles — all without ever reaching code generation. `clangd` (built directly on Clang's front-end), `rust-analyzer`, and `gopls` are all, in effect, "compilers with the back end removed and an editor bolted on instead." This is a direct, practical payoff of the front-end/back-end separation described above: a front-end built for compilation can be repurposed wholesale for tooling.

2.4 Comparing Real-World IRs: LLVM IR, JVM Bytecode, and WebAssembly

[beyond DB] LLVM IR, JVM bytecode, and WebAssembly are all, loosely, "intermediate, machine-independent instruction sets sitting between source code and a real CPU" — which is why students often lump them together as interchangeable "bytecode." They are not interchangeable: each was designed with a different primary goal in mind, and that goal shapes how high- or low-level it is, what it assumes about memory and objects, and what it is actually good for.

	LLVM IR	**JVM Bytecode**	**WebAssembly (Wasm)**
Primary design goal	A reusable *compiler middle-end*: one optimizer, shared by many front-ends and back-ends	*Portability* ("write once, run anywhere") for Java-family, object-oriented, garbage-collected languages	*Safe, sandboxed* execution of code from an untrusted source (originally: the web), at near-native speed
Abstraction level	Low — close to assembly, but target-independent; typed and SSA-based	Medium-to-high — bakes in classes, interfaces, virtual method dispatch, exception tables	Low — a compact, portable, stack-based instruction set, close to assembly
Assumes an object model / GC?	No. No built-in notion of classes or garbage collection; each front-end supplies its own	Yes. Assumes the JVM supplies garbage collection and an object model	No, in Wasm's core spec. A separate, later "GC proposal" adds optional managed references
Control flow	Ordinary basic blocks and branches, the same shape as real machine code	Ordinary jumps/branches; the bytecode *verifier* checks type-safety, not control-flow shape	**Structured only** — blocks, loops, ifs; no arbitrary `goto`, so a validator can check it in a single linear pass
Usually executed directly?	No — almost always compiled further, to native code; not normally shipped as the final artifact users run	Yes — interpreted and/or JIT-compiled directly by a JVM	Yes — interpreted and/or JIT/AOT-compiled directly by a Wasm runtime
Best fit for	Being a shared layer that *very different* source languages (systems, functional, managed, ...) can all lower into	Languages that already fit the JVM's object/GC model well (Kotlin, Scala, Clojure, Groovy)	Any source language willing to compile to a small, safe, sandboxed core – often reached *via* LLVM's own WebAssembly back-end

Why LLVM IR's Low Level Is a Feature, Not a Limitation

LLVM IR deliberately does *not* assume any particular memory-management strategy or object model. That is exactly why it can serve as the shared middle layer for languages with wildly different runtime models: C's manual memory management, Rust's ownership/borrow-checked memory, Swift's automatic reference counting, and Julia's dynamic dispatch all lower into the same LLVM IR, because none of those choices had to be baked into the IR itself – each front-end encodes its own memory-management decisions *as ordinary LLVM IR instructions*. JVM bytecode makes the opposite trade-off: by assuming garbage collection and an object model up front, it becomes an excellent, low-friction target for languages that already want exactly that (Kotlin, Scala) but an awkward one for a language like C that manages memory itself. Neither design is "better" in general – they optimize for different things, which is precisely the point of this comparison.

Example: WebAssembly as "Just Another LLVM Back-End"

Because LLVM already has back-ends for x86-64, AArch64, and RISC-V, adding a WebAssembly back-end fits the exact same $M + N$ story from earlier in this lecture: any existing LLVM front-end (Clang for C/C++, `rustc` for Rust) can already produce WebAssembly simply by selecting the `wasm32` target, with *no* changes to the front-end or to any optimization pass. This is why, in practice, a great deal of WebAssembly in the wild is not hand-written but is the output of compiling C, C++, or Rust source through ordinary LLVM tooling (e.g. Emscripten for C/C++) – WebAssembly is simultaneously a stand-alone IR in its own right *and* one more entry in LLVM's list of back-end targets.

Exam Focus

Do not confuse the two axes this section and the earlier "kinds of IR" discussion test separately: *how low-level is it, and why* (this section) versus *when is it executed relative to run time* (Lecture **??**'s AOT/interpreter/JIT distinction). LLVM IR and WebAssembly are both low-level, but for different reasons – LLVM IR is low-level so that generating real machine code from it is easy and so that no language's runtime assumptions get baked in; WebAssembly is low-level *and* structured so that it can be validated quickly and executed safely in a sandbox. JVM bytecode is higher-level than both, by design, because portability for one specific family of managed languages – not raw compiler-infrastructure reuse, and not sandboxed safety for arbitrary source languages – was its original goal.

2.5 MLIR: A Framework for Building Compiler IRs

MLIR ("Multi-Level Intermediate Representation") is a compiler-infrastructure project started at Google around 2019 and merged into the official LLVM monorepo as one of its sub-projects in 2020, first described publicly in [MLIR]. It is worth understanding alongside LLVM IR for the same reason the previous section compared LLVM IR, JVM bytecode, and WebAssembly: people often assume "MLIR" is just another name for "LLVM IR, but newer," when in fact the two solve different problems and sit at different points in a compiler's pipeline.

2.5.1 The Problem MLIR Was Built to Solve

LLVM IR is deliberately low-level: typed, in SSA form, organized into ordinary basic blocks – close to assembly, as §2.4 described. That is exactly what makes it a good shared target for many different *source* languages once they have already been reduced to simple operations and control flow. It is a poor fit, however, for representing a program *before* that reduction has happened – for example, a matrix multiplication expressed as a single high-level tensor operation, or a nested loop that a compiler still intends to tile, fuse, or vectorize as a unit. Lowering straight to LLVM IR forces that structure to be flattened away immediately, before any optimization pass gets a chance to use it.

Before MLIR existed, projects that needed this kind of higher-level representation simply built their own IR from scratch: TensorFlow's XLA had its own IR for tensor computations, Swift had SIL, Rust had MIR, and so on. Each of these reimplemented the same underlying machinery – a verifier, a textual printer and parser, a pass manager, a way to track source locations for error messages – that LLVM already provides for LLVM IR, but for their own, unrelated IR. MLIR's goal is to give every such project a shared framework for this machinery, so that building a new, domain-specific IR is a matter of defining new operations rather than rebuilding compiler infrastructure from the ground up.

2.5.2 Core Ideas

Dialects. Instead of one fixed instruction set, MLIR defines an extensible mechanism for registering **dialects** – namespaced families of operations, types, and attributes. The `arith` dialect has operations like `arith.addi`; the `linalg` dialect has structured, high-level operations such as matrix multiplication; the `scf` dialect ("structured control flow") has `for` and `while` loops as single operations, rather than as jumps between basic blocks; and the `llvm` dialect defines operations that directly mirror LLVM IR. Crucially, operations from several different dialects can appear together in the very same function – there is no requirement to fully translate out of one dialect before using another.

Progressive lowering. A program does not jump from a high-level dialect straight to machine code in one step. It is lowered dialect by dialect: a high-level tensor operation becomes a structured loop, the structured loop becomes ordinary branching control flow, and that, in turn, becomes the `llvm` dialect. Each of these steps is a small, independently checkable transformation, and different compilers can choose to stop lowering at whichever level is most useful for a given optimization – a loop-tiling pass wants to run before loops are flattened into branches, for instance, while register allocation only makes sense long after that point.

Regions. An ordinary LLVM IR function is a flat sequence of basic blocks connected by branches – there is no notion of one block being "inside" another. An MLIR operation, by contrast, can carry a nested **region** containing its own blocks. A loop or a function body is therefore naturally represented as one operation containing a region, preserving its nested structure for as long as that structure remains useful, instead of being flattened into a control-flow graph immediately.

2.5.3 How MLIR Differs From LLVM IR

	LLVM IR	**MLIR**
What it is	One fixed intermediate representation	A framework for defining many intermediate representations (dialects)
Abstraction level	Low and fixed – close to assembly	Whatever a given dialect needs – high, low, or several levels mixed in one module
Control-flow structure	Flat basic blocks connected by branches	Operations can carry nested regions, so loops and other structure can stay nested until a pass chooses to flatten them
Role in a pipeline	The representation code is lowered *to*, immediately before LLVM's own optimizer and back-ends take over	Sits *upstream* of LLVM IR; its own `llvm` dialect is what eventually converts into ordinary LLVM IR

The last row is the point most often missed: MLIR does not replace LLVM. A program written using MLIR's higher-level dialects is progressively lowered, within MLIR, down to the `llvm` dialect; from there it is translated into ordinary LLVM IR and handed to the exact same optimizer passes and back-ends already described in §2.3 and §2.4. MLIR extends the *front* of the pipeline – giving compiler writers a place to do domain-specific reasoning (tensor shapes, loop tiling, hardware-specific fusion) before code is flattened down to LLVM IR – rather than replacing anything downstream of it.

2.5.4 Where MLIR Is Used

MLIR's dialect mechanism has made it attractive well beyond its original use case. Notable adopters include Google's XLA and IREE (compiling machine-learning computation graphs to CPUs, GPUs,

and custom accelerators), PyTorch's Torch-MLIR, Triton (a language for writing GPU kernels), CIRCT (an MLIR-based toolchain for hardware design, playing a role similar to parts of a Verilog toolchain), and Flang, LLVM's own Fortran front end, which lowers through an MLIR dialect called FIR before reaching LLVM IR (see [MLIR, MLIR-DOCS] for further reading).

2.6 Self-Check Questions

Practice – Not Graded

1. Place the following in correct pipeline order: semantic analysis, linking, assembly, lexical analysis, loading, syntax analysis, preprocessing, code generation.
2. A program calls a function defined in a library it never `#includes` a declaration for, but the linker still resolves the call successfully because the symbol name matches at link time. Which phase would have caught this as an error if the declaration truly were missing, and why didn't it?
3. Explain, in the M-languages/N-targets argument, why a shared IR turns an $M \times N$ problem into an $M + N$ problem. What is the "shared IR" in LLVM's case?
4. Why does dynamic linking trade smaller executables for slower startup, while static linking does the opposite?
5. `clangd` never performs code generation. Which phases of the pipeline does it still need to run, and why are those enough to power features like "jump to definition"?
6. For the `position` example in §2.1.1, write out what the three-address code would look like if `60` were replaced by `count`, an already-integer variable (so no `inttoreal` conversion is needed). Which of the six phases changes as a result, and which stay exactly the same?
7. JVM bytecode assumes garbage collection; LLVM IR does not. Explain why this makes JVM bytecode a natural fit for Kotlin but a poor natural fit for a language like C, and explain why WebAssembly's core spec makes the same choice as LLVM IR here rather than the JVM's.
8. For the statement `count = count + step`, list its lexemes, give the `<token-name, attribute-value>` pair for each one, and sketch the symbol table entries lexical analysis would create (assuming `count` and `step` are both integers).
9. Draw the syntax tree for `count = count + step` in the same style as the `position` example's tree, and identify which single node the semantic analyzer would need to annotate if `step` were declared `real` instead of integer.
10. For each of the following, classify the mistake as a lexical, syntax, or semantic error, and justify your answer in one sentence: (a) `int x = 5$;` (b) `if (x > 0 { y = 1; }` (missing a closing parenthesis) (c) calling `compute(x, y)` when `compute` was declared to take only one parameter.

Key Takeaways

- A compiler's six classical phases, in order: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, code generation.
- These split into an **analysis** half (front end: builds an IR and symbol table) and a **synthesis** half (back end: builds the target program from that IR).
- The `position = initial + rate * 60` example shows all six phases acting on one tiny statement: tokens → syntax tree → annotated tree (type conversion inserted) → three-address code → optimized three-address code (constant folded) → target assembly.

- A **token** is an abstract `<name, attribute>` pair emitted for a recognized **lexeme** (the actual matched source text). A **symbol table** records one entry per declared identifier and is read from and written to by every phase – not a phase itself.
- **Lexical, syntax, and semantic errors** are caught by three different phases, in that order: a lexical error means no token pattern matches at all (e.g. an illegal character); a syntax error means every token is valid but their arrangement breaks the grammar (e.g. a missing semicolon); a semantic error means the program is grammatically fine but violates a meaning-level rule the language enforces (e.g. an undeclared identifier). A misspelled keyword is usually *not* a lexical error, since it is still a valid identifier lexeme.
- The **front-end/middle-end/back-end** split (language-specific $\rightarrow$ language-and-target-agnostic $\rightarrow$ target-specific) is what lets one shared IR (like LLVM IR) turn an $M \times N$ language/target problem into an $M + N$ problem.
- **LLVM IR, JVM bytecode, and WebAssembly** are all machine-independent instruction sets but were built for three different goals – reusable compiler infrastructure, portability for GC'd object-oriented languages, and safe sandboxed execution, respectively – and that goal explains how high- or low-level each one is and what it assumes about memory and objects.
- **MLIR** is not a fixed IR but a framework for defining many **dialects** at different abstraction levels, progressively lowered down to LLVM IR – it extends the front of the pipeline rather than replacing anything downstream of it.
- Front-ends have value beyond compilation: language servers (`clangd`, `rust-analyzer`) repurpose the same lexer/parser/semantic-analyzer pipeline for IDE tooling, without ever reaching code generation.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[PP] Scott, M. L. *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann.

[MLIR] Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., and Zinenko, O. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. arXiv:2002.11054, 2020.

[MLIR-DOCS] The MLIR Project. *Official Documentation*. https://mlir.llvm.org/